

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Отчет по лабораторной работе № 6
По дисциплине: «Современный платформы программирования»
Специальность: «Программная инженерия»

Выполнил:
О.А. Козюта
Студент группы ПО-13
Проверил:
А.Д. Кулик
Преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`)

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart` `@pytest.fixture`

```
def empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1.

Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

8) Напишите метод `int levenshteinDistance(String s, String t)` рассчитывающий расстояние Ливенштейна для двух строк. Расстояние Ливенштейна между двумя строками – это то количество посимвольных трансформаций необходимое, что бы превратить одну строку в другую.

Спецификация метода:

`levenshteinDistance (None, None) = TypeError`

`levenshteinDistance (None, *) = -1`

`levenshteinDistance (*, None) = -1`

`levenshteinDistance ("", "") = 0`

`levenshteinDistance ("", "a") = 1`

`levenshteinDistance (" aaapppp ", "") = 7`

`levenshteinDistance (" frog ", " fog ") = 1`

`levenshteinDistance ("fly", " ant ") = 3`

`levenshteinDistance (" elephant ", " hippo ") = 7`

`levenshteinDistance (" hippo ", " elephant ") = 7`

`levenshteinDistance (" hippo ", " zzzzzzzz ") = 8`

`levenshteinDistance (" hello ", " hallo ") = 1`

shopping.py

```
class Cart:
```

```
    """Shopping cart with items and prices."""
```

```
    def __init__(self):
```

```
        self.items = {}
```

```
    def add_item(self, name: str, price: float, quantity: int = 1):
```

```
        """Add an item or increase quantity."""
```

```
        if price <= 0 or quantity <= 0:
```

```
            raise ValueError("Price and quantity must be positive")
```

```
        if name in self.items:
```

```
            self.items[name]["quantity"] += quantity
```

```
        else:
```

```
            self.items[name] = {"price": price, "quantity": quantity}
```

```

def remove_item(self, name: str, quantity: int = 1):
    """Remove given quantity of an item."""
    if name not in self.items:
        raise KeyError(f"Item {name} not in cart")
    if quantity <= 0:
        raise ValueError("Quantity must be positive")
    if self.items[name]["quantity"] < quantity:
        raise ValueError(f"Cannot remove {quantity}, only {self.items[name]['quantity']}
available")
    self.items[name]["quantity"] -= quantity
    if self.items[name]["quantity"] == 0:
        del self.items[name]

def total(self) -> float:
    """Calculate total price."""
    return sum(item["price"] * item["quantity"] for item in self.items.values())

def clear(self):
    """Remove all items."""
    self.items.clear()

def __len__(self):
    return sum(item["quantity"] for item in self.items.values())

```

test_shopping.py

```

import pytest
from shopping import Cart

@pytest.fixture
def empty_cart():
    """Returns a new empty cart."""
    return Cart()

def test_add_item(empty_cart):
    empty_cart.add_item("apple", 1.5, 2)
    assert len(empty_cart) == 2
    assert empty_cart.total() == 3.0

def test_add_duplicate_item(empty_cart):
    empty_cart.add_item("banana", 2.0, 1)
    empty_cart.add_item("banana", 2.0, 3)
    assert len(empty_cart) == 4
    assert empty_cart.total() == 8.0

def test_add_invalid_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("bad", -1.0, 1)

def test_remove_item(empty_cart):
    empty_cart.add_item("orange", 3.0, 5)

```

```

empty_cart.remove_item("orange", 2)
assert len(empty_cart) == 3
assert empty_cart.total() == 9.0

def test_remove_too_much(empty_cart):
    empty_cart.add_item("pear", 1.0, 1)
    with pytest.raises(ValueError):
        empty_cart.remove_item("pear", 2)

def test_remove_missing_item(empty_cart):
    with pytest.raises(KeyError):
        empty_cart.remove_item("nonexistent")

def test_clear(empty_cart):
    empty_cart.add_item("a", 1, 2)
    empty_cart.add_item("b", 2, 3)
    empty_cart.clear()
    assert len(empty_cart) == 0
    assert empty_cart.total() == 0.0

```

lab1_functions.py

```

def unique_numbers(numbers):
    """Return list of unique numbers from input list (preserve order of first appearance)."""
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    seen = set()
    result = []
    for n in numbers:
        if n not in seen:
            seen.add(n)
            result.append(n)
    return result

def add_binary(a: str, b: str) -> str:
    """Return sum of two binary strings as binary string."""
    if not isinstance(a, str) or not isinstance(b, str):
        raise TypeError("Both arguments must be strings")
    # Convert to integer, add, then back to binary without '0b'
    return bin(int(a, 2) + int(b, 2))[2:]

```

test_lab1_functions.py

```

import pytest
from lab1_functions import unique_numbers, add_binary

# Тесты для unique_numbers
def test_unique_numbers_trivial():
    assert unique_numbers([]) == []
    assert unique_numbers([1]) == [1]
    assert unique_numbers([1, 2, 3]) == [1, 2, 3]

```

```
def test_unique_numbers_with_duplicates():
    assert unique_numbers([1, 2, 2, 3, 3, 3]) == [1, 2, 3]
    assert unique_numbers([5, 5, 5]) == [5]
```

```
def test_unique_numbers_type_error():
    with pytest.raises(TypeError):
        unique_numbers("not a list")
```

Тесты для add_binary

```
def test_add_binary_example():
    assert add_binary("11", "1") == "100"
```

```
def test_add_binary_zero():
    assert add_binary("0", "0") == "0"
```

```
def test_add_binary_carry():
    assert add_binary("111", "1") == "1000"
    assert add_binary("1010", "1011") == "10101"
```

```
def test_add_binary_with_leading_zeros():
    assert add_binary("001", "01") == "10"
```

```
def test_add_binary_type_error():
    with pytest.raises(TypeError):
        add_binary(123, "1")
```

levenshtein.py

```
def levenshtein_distance(s, t):
```

```
    """
```

Calculate Levenshtein distance between two strings.

Special cases:

- If both are None -> raises TypeError
- If one is None -> returns -1
- Otherwise returns integer distance.

```
    """
```

if s is None and t is None:

```
    raise TypeError("Both arguments cannot be None")
```

if s is None or t is None:

```
    return -1
```

Convert to strings in case of other types? Spec expects strings.

```
s = str(s)
```

```
t = str(t)
```

```
m, n = len(s), len(t)
```

```
dp = [[0] * (n + 1) for _ in range(m + 1)]
```

```
for i in range(m + 1):
```

```
    dp[i][0] = i
```

```
for j in range(n + 1):
```

```

dp[0][j] = j

for i in range(1, m + 1):
    for j in range(1, n + 1):
        if s[i - 1] == t[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]
        else:
            dp[i][j] = 1 + min(dp[i - 1][j],    # deletion
                               dp[i][j - 1],    # insertion
                               dp[i - 1][j - 1]) # substitution
    return dp[m][n]

```

test_levenshtein.py

```

import pytest
from levenshtein import levenshtein_distance

def test_both_none():
    with pytest.raises(TypeError):
        levenshtein_distance(None, None)

def test_first_none():
    assert levenshtein_distance(None, "abc") == -1

def test_second_none():
    assert levenshtein_distance("abc", None) == -1

def test_empty_strings():
    assert levenshtein_distance("", "") == 0

def test_empty_vs_string():
    assert levenshtein_distance("", "a") == 1
    assert levenshtein_distance("", "aaapppp") == 7

def test_spaces_examples():
    # Spec tests with spaces (note: actual strings include spaces)
    assert levenshtein_distance(" frog ", " fog ") == 1
    assert levenshtein_distance("fly", " ant ") == 3
    assert levenshtein_distance(" elephant ", " hippo ") == 7
    assert levenshtein_distance(" hippo ", " elephant ") == 7
    assert levenshtein_distance(" hippo ", " zzzzzzzz ") == 8

def test_hello_hallo():
    assert levenshtein_distance(" hello ", " hallo ") == 1

def test_normal_words():
    assert levenshtein_distance("kitten", "sitting") == 3
    assert levenshtein_distance("flaw", "lawn") == 2

```

Вывод: освоены приемы тестирования кода на примере использования пакета pytest.